

GUÍA PRÁCTICA · NEIRO AGENCY

Tu propio MCP, conectado a tu negocio

La ruta técnica completa: estructura del proyecto, tu primera tool, cómo probarlo local, conectarlo a Claude, y ponerlo a trabajar con los sistemas reales de tu negocio, con permisos mínimos y control real sobre lo que toca.

Marcos Vargas · IA · Sistemas · Automatización

LO QUE APRENDERÁS

- 01** Qué es un MCP y por qué le da a Claude acceso directo a tu negocio
- 02** El setup técnico: SDK, estructura y configuración
- 03** Cómo definir tu primera tool con schema y manejo de errores
- 04** Cómo probarlo local con MCP Inspector antes de conectarlo
- 05** Cómo conectarlo a Claude Code y Claude Desktop
- 06** Ponerlo en producción: permisos mínimos, revocables y logueados

SECCIÓN 01 · EL CONTEXTO

La vía más técnica. También la que más engancha con tu negocio.

Un servidor MCP (Model Context Protocol) es lo que le permite a un agente de IA usar una herramienta tuya como si fuera nativa: consultar tu inventario, revisar tu agenda, leer o actualizar un pedido, ejecutar una acción concreta sobre un sistema real. Es la pieza que le falta a la mayoría de las automatizaciones con n8n o chatbots sueltos: que el agente pueda actuar sobre los datos reales de tu negocio, no solo sobre lo que copiás y pegás en un chat.

Es la vía de esta guía que menos se parece a comprar una herramienta más y más se parece a que la IA trabaje adentro de tu negocio: requiere saber programar (o dejar que Claude Code te ayude a hacerlo), pero no necesitás un equipo ni infraestructura compleja para el primer MCP. Y a diferencia de un dashboard más, esto le da a Claude una forma directa de leer y tocar tu negocio, dentro de los límites que vos definís.

Qué es gratis y qué no: escribir el código del MCP con Claude Code pide un plan pago de Claude (Pro desde 20 dólares al mes) o, si preferís no pagar una suscripción fija, créditos de la API por consumo desde Claude Console, sin mínimo mensual. El SDK (`@modelcontextprotocol/sdk`) y el MCP Inspector para probarlo local son gratis y de código abierto, no pagás nada extra por esa parte.

Nota: el precio de los planes de Claude puede variar según la región, el tipo de cambio o el momento en que estés leyendo esto. Revisá la tarifa vigente en el sitio oficial antes de presupuestar.

65-130h

DESARROLLO ESTIMADO PARA TU
PRIMER MCP, SI LO HACÉS DE
CERO

3

PREGUNTAS A RESPONDER ANTES
DE CONECTAR CUALQUIER MCP A
UN SISTEMA REAL: QUÉ DATOS
TOCA, QUÉ ACCIÓN EJECUTA, QUÉ
PLATA MUEVE

SECCIÓN 02 · SETUP TÉCNICO

Estructura del proyecto y SDK.

ESTRUCTURA RECOMENDADA

```
mi-mcp-server/
├─ src/
│  ├─ index.ts          (punto de entrada)
│  ├─ constants.ts
│  ├─ types.ts
│  ├─ tools/            (una tool por archivo)
│  ├─ services/         (llamadas a tu API o base de datos)
│  └─ schemas/          (validación con zod)
├─ package.json
└─ tsconfig.json
```

INSTALAR EL SDK

```
npm install @modelcontextprotocol/sdk zod axios
npm install -D typescript @types/node tsx
```

CONFIGURACIÓN TYPESCRIPT

Usá `"module": "Node16"` con `"moduleResolution": "Node16"` en tu `tsconfig.json` para trabajar con ES Modules. Los imports relativos entre tus propios archivos necesitan la extensión `.js` aunque el archivo original sea `.ts`.

PUNTO CRÍTICO

Usá `console.error()` para tus logs, nunca `console.log()`. El transporte stdio usa stdout exclusivamente para el protocolo MCP, si mandás un log ahí, rompés la comunicación con el agente.

SECCIÓN 03 · TU PRIMERA TOOL

Lo que el agente va a poder usar.

Cada tool que registrás es una función que el agente puede decidir llamar. Cuatro cosas importan más que el código en sí:

- **Nombre con prefijo claro** (ej. `inventario_buscar_producto`), para que no choque con tools de otros MCP conectados.
- **Descripción exhaustiva**: es lo que el modelo lee para decidir si usar tu tool o no. Una descripción vaga significa que el agente la ignora o la usa mal.
- **Schema con Zod y `.strict()`**: valida automáticamente lo que el agente manda, y rechaza campos que no pediste.
- **Errores accionables**: si algo falla, el mensaje de error tiene que decirle al agente qué hacer distinto, no solo "error 500".

PATRÓN BÁSICO

```
server.registerTool(  
  "inventario_buscar_producto",  
  { description: "...", inputSchema: schemaZod.strict() },  
  async (params) => {  
    // tu lógica: llamada a API, consulta a base de datos, etc.  
  }  
);
```

SECCIÓN 04 · PROBARLO LOCAL

MCP Inspector, antes de conectar nada.

Nunca conectes un MCP a un sistema real del negocio sin probarlo aislado primero.

ruta de puesta en marcha

1. Compilá tu proyecto: `npm run build`.
2. Lanzá el inspector: `npx @modelcontextprotocol/inspector node dist/index.js`.
3. Se abre una interfaz en `http://localhost:6274` donde podés ejecutar cada tool manualmente y ver el resultado exacto que recibiría el agente.
4. Probá casos límite: parámetros vacíos, valores fuera de rango, para confirmar que tus errores accionables funcionan.

SECCIÓN 05 · CONECTARLO A CLAUDE

De tu máquina a un agente real.

CLAUDE CODE

```
claude mcp add mi-mcp-server -- node /ruta/absoluta/dist/index.js
```

O edita el archivo `.claude.json` manualmente con la configuración stdio si preferís hacerlo a mano.

CLAUDE DESKTOP

Edita el archivo de configuración según tu sistema operativo:

- **macOS:** `~/Library/Application Support/Claude/claude_desktop_config.json`
- **Windows:** `%APPDATA%\Claude\claude_desktop_config.json`

VERIFICÁ LA VERSIÓN

El SDK de MCP, el Inspector y los comandos de Claude Code evolucionan rápido. Antes de copiar un comando o una ruta, confirmalo en la documentación oficial (modelcontextprotocol.io y code.claude.com/docs), sobre todo si tu versión es más nueva que la de esta guía.

DOS TRANSPORTES DISPONIBLES

Stdio: local, sin configuración de red, ideal para uso personal o pruebas. **Streamable HTTP:** remoto, soporta múltiples clientes a la vez, es el que necesitás si varias personas de tu equipo lo van a usar a la vez, o si otro sistema tuyo lo va a llamar de forma remota (`TRANSPORT=http` `PORT=3000`).

SECCIÓN 06 · PONERLO EN PRODUCCIÓN

Tratalo como a un empleado nuevo: permisos mínimos, revocables y logueados.

El error más caro con un MCP no es de código, es de permisos. Un agente con acceso de escritura a tu sistema real puede equivocarse, alucinar un parámetro, o simplemente hacer algo que no le pediste con el permiso que sí le diste. La pregunta que importa no es "qué le pedí que hiciera", es "qué puede tocar si algo sale mal": qué datos lee, qué acción puede ejecutar, qué plata puede mover.

RUTA DE PUESTA EN MARCHA

1. **Credenciales mínimas:** creá un usuario o API key aparte para el MCP, con acceso solo a lo que esa tool necesita tocar. Nunca lo conectés con el admin general de tu sistema.
2. **Empezá en solo lectura:** leer inventario, leer agenda, leer un pedido. Subí a permisos de escritura (crear, modificar, cobrar) solo cuando ya viste cómo se comporta en lectura.
3. **Que sea revocable en un clic:** tenés que poder cortarle el acceso al MCP sin rotar las credenciales de todo el negocio.
4. **Logueá cada llamada:** qué tool se ejecutó, con qué parámetros, cuándo. Si algo sale mal, necesitás poder ver exactamente qué tocó.
5. **Definí un kill switch:** una forma de apagar el MCP completo, o una sola tool, sin afectar el resto del negocio.

PUNTO CRÍTICO

Razoná desde lo que el MCP PUEDE tocar, no desde lo que le pediste que hiciera. Un prompt le pide una cosa puntual; un bug, una mala interpretación o un uso indebido puede hacer que use el permiso que le diste para otra cosa distinta.

Cada MCP que conectés a un sistema real debería llevar, aunque sea en un documento corto, un registro de qué automatiza, qué puede tocar y cómo se revoca. No hace falta un contrato de diez páginas, con una página basta, pero que exista.

REALIDAD

La confianza se gana por fases, con evidencia, no se asume desde el primer día. Dale acceso de lectura, medí un tiempo cómo se comporta, y solo ahí subí el nivel de permiso. Un MCP se termina de construir en un fin de semana; ganarse el permiso de tocar plata o datos sensibles del negocio se termina de construir con evidencia, no con una sola sesión de código.

SECCIÓN 07 · PRÓXIMA ACCIÓN

Armá tu primera tool esta semana.

IA aplicada a sistemas propios.

PRÓXIMA ACCIÓN

No empieces dándole a Claude acceso de escritura a tu sistema completo. Empezá por detectar un problema chico y específico que ya resolvés a mano (buscar un producto, revisar una fecha, consultar un estado) y convertilo en una sola tool de solo lectura. Probala local con el Inspector antes de pensar en subir el nivel de permiso.